

Sim. Vou manter a **mesma estrutura didática da aula Java**, mas fazendo a transposição para uma aplicação web em **PHP procedural + MySQL + Bootstrap + JavaScript**, com um projeto de **Cadastro de Alunos** e CRUD completo.

A principal diferença conceitual será:

```
JAVA + SWING
  ↓
Aplicação desktop
  ↓
JDBC
  ↓
MySQL
```

versus:

```
PHP + HTML + Bootstrap + JavaScript
  ↓
Aplicação web
  ↓
MySQLi/PDO
  ↓
MySQL
```

Como você pediu especificamente **PHP procedural**, vou evitar Laravel, orientação a objetos e frameworks no código principal.

Aula Completa — PHP Procedural + MySQL + Bootstrap + JavaScript

1. Objetivos

Ao final da aula, o aluno deverá ser capaz de:

- criar uma aplicação PHP;
- configurar um servidor web local;
- criar banco e tabela MySQL;
- conectar PHP ao MySQL;
- utilizar PHP procedural;
- executar `INSERT`, `SELECT`, `UPDATE` e `DELETE`;
- criar formulários HTML;
- utilizar Bootstrap para estilização;
- utilizar JavaScript para validação;
- utilizar `fetch()` para comunicação assíncrona;
- apresentar registros em tabela;
- criar um CRUD completo;

- compreender a diferença entre código executado no servidor e no navegador.

2. Projeto

Vamos desenvolver:

Sistema de Cadastro de Alunos

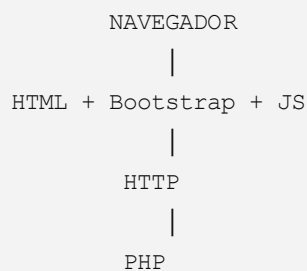
A aplicação terá:

CADASTRO DE ALUNOS				
Nome				
[_____]				
E-mail				
[_____]				
Curso				
[_____]				
[Cadastrar] [Limpar]				
ID	Nome	E-mail	Curso	Ações
1	João	...	Info	Editar Excluir

3. Arquitetura

Diferentemente do Java Swing, não teremos uma aplicação desktop.

Teremos:



Podemos dividir o projeto:

```
cadastro-alunos/  
|  
├─ index.php  
├─ salvar.php  
├─ editar.php  
├─ excluir.php  
├─ conexao.php  
├─ buscar.php  
├─ script.js  
|  
└─ assets/  
    └─ bootstrap/
```

Mais adiante podemos evoluir para:

```
cadastro-alunos/  
|  
├─ index.php  
├─ conexao.php  
|  
├─ aluno/  
|   ├─ salvar.php  
|   ├─ editar.php  
|   └─ excluir.php  
|   └─ buscar.php  
|  
├─ css/  
|   └─ estilo.css  
|  
└─ js/  
    └─ script.js
```

4. Pré-requisitos

O aluno deve conhecer:

PHP

PHP

```
<?php
```

```
$nome = "João";  
  
echo $nome;
```

Variáveis:

PHP

```
$nome = "Maria";  
$idade = 20;
```

Condicional:

PHP

```
if ($idade >= 18) {  
    echo "Maior de idade";  
}
```

Formulários:

PHP

```
$_POST  
$_GET
```

HTML

HTML

```
<form>  
    <input>  
    <button>  
</form>
```

SQL

SQL

```
INSERT  
SELECT  
UPDATE  
DELETE
```

5. Banco de dados

Crie:

SQL

```
CREATE DATABASE escola;
```

Depois:

SQL

```
USE escola;
```

Tabela:

SQL

```
CREATE TABLE alunos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(150),  
    curso VARCHAR(100)  
);
```

Dados iniciais:

SQL

```
INSERT INTO alunos  
(nome, email, curso)  
VALUES  
(  
    'João Silva', 'joao@email.com', 'Informática',  
    'Maria Souza', 'maria@email.com', 'Administração');
```

Teste:

SQL

```
SELECT * FROM alunos;
```

6. Conexão PHP + MySQL

Crie:

conexao.php

Código:

PHP

```
<?php

$servidor = "localhost";
$usuario = "root";
$senha = "";
$banco = "escola";

$conexao = mysqli_connect(
    $servidor,
    $usuario,
    $senha,
    $banco
);

if (!$conexao) {

    die(
        "Erro na conexão: "
        . mysqli_connect_error()
    );
}

mysqli_set_charset($conexao, "utf8mb4");
```

7. Entendendo a conexão

Temos:

```
localhost
  ↓
MySQL
  ↓
escola
```

E no PHP:

PHP

```
mysqli_connect(
    $servidor,
    $usuario,
    $senha,
```

```
$banco  
);
```

O resultado é armazenado:

```
PHP  
  
$conexao
```

Essa variável será utilizada pelas operações SQL.

8. Primeiro teste

Crie:

```
teste.php
```

```
PHP  
  
<?php  
  
require_once "conexao.php";  
  
echo "Banco conectado com sucesso!";
```

Abra no navegador:

```
http://localhost/cadastro-alunos/teste.php
```

Resultado:

```
Banco conectado com sucesso!
```

9. PHP procedural

Aqui temos uma diferença importante em relação à aula Java.

No Java tínhamos:

```
Java  
  
Aluno aluno = new Aluno();
```

```
AlunoDAO dao = new AlunoDAO();
```

No PHP procedural podemos trabalhar diretamente com variáveis:

PHP

```
$nome = $_POST["nome"];  
$email = $_POST["email"];  
$curso = $_POST["curso"];
```

E executar o SQL.

Isso facilita a entrada inicial do aluno no desenvolvimento web, embora posteriormente seja interessante mostrar como essa aplicação poderia ser organizada com funções, classes e MVC.

10. INSERT

Crie:

salvar.php

Código:

PHP

```
<?php  
  
require_once "conexao.php";  
  
$nome = $_POST["nome"];  
$email = $_POST["email"];  
$curso = $_POST["curso"];  
  
$sql = "  
    INSERT INTO alunos  
    (nome, email, curso)  
    VALUES  
    ('$nome', '$email', '$curso')  
";  
  
$resultado = mysqli_query(  
    $conexao,  
    $sql  
);  
  
if ($resultado) {
```



```
header("Location: index.php");

} else {

    echo "Erro: "
        . mysqli_error($conexao);
}
```

Aqui o aluno já consegue visualizar:

FORMULÁRIO

↓

POST

↓

PHP

↓

SQL

↓

MYSQL

11. Um problema importante

O código anterior funciona, mas possui um problema sério:

PHP

'\$nome'

está sendo concatenado diretamente no SQL.

Isso pode permitir **SQL Injection**.

Portanto, depois de demonstrar o funcionamento procedural básico, eu faria uma pausa pedagógica e mostraria:

"Funcionar não significa estar correto ou seguro."

Para a versão real, devemos utilizar `mysqli_prepare()`.

12. INSERT seguro

PHP

```

<?php

require_once "conexao.php";

$nome = $_POST["nome"];
$email = $_POST["email"];
$curso = $_POST["curso"];

$sql = "
    INSERT INTO alunos
    (nome, email, curso)
    VALUES (?, ?, ?)
";

$stmt = mysqli_prepare(
    $conexao,
    $sql
);

mysqli_stmt_bind_param(
    $stmt,
    "sss",
    $nome,
    $email,
    $curso
);

if (mysqli_stmt_execute($stmt)) {

    header("Location: index.php");

    exit;

} else {

    echo "Erro ao cadastrar.";

}

```

Isso cria uma excelente comparação com a aula Java:

Java	PHP
PreparedStatement	mysqli_prepare()
setString()	mysqli_stmt_bind_param()
executeUpdate()	mysqli_stmt_execute()

13. Criando a interface

Agora entra o Bootstrap.

No `index.php`:

HTML

```
<!DOCTYPE html>

<html lang="pt-BR">

<head>

  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1"
  >

  <title>Cadastro de Alunos</title>

  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet"
```

14. Formulário Bootstrap

Dentro do container:

HTML

```
<div class="card">

  <div class="card-header">
    Novo aluno
  </div>

  <div class="card-body">

    <form
      action="salvar.php"
      method="POST"
      id="formAluno"
    >

      <div class="mb-3">
```

```
<label class="form-label">
```

Nome

15. SELECT

Agora precisamos mostrar os alunos.

No `index.php`, depois do formulário:

PHP

```
<?php

require_once "conexao.php";

$sql = "
    SELECT *
    FROM alunos
    ORDER BY nome
";

$resultado = mysqli_query(
    $conexao,
    $sql
);
?>
```

16. Tabela Bootstrap

HTML

```
<div class="card mt-4">

    <div class="card-header">
        Alunos cadastrados
    </div>

    <div class="card-body">

        <table class="table table-striped table-hover">

            <thead>

                <tr>
                    <th>ID</th>
                    <th>Nome</th>
```

```
<th>E-mail</th>
<th>Curso</th>
```

17. Conceito importante: PHP dentro do HTML

Aqui o aluno começa a entender uma característica fundamental do PHP:

PHP

```
<?php ?>
```

permite executar código no servidor.

Por exemplo:

PHP

```
<?php while ($aluno = mysqli_fetch_assoc($resultado)): ?>
```

E depois:

PHP

```
<?php endwhile; ?>
```

O navegador **não recebe PHP**.

Recebe HTML.

```
PHP
↓
Servidor
↓
HTML
↓
Navegador
```

18. UPDATE

Crie:

```
editar.php
```

Primeiro recuperamos o ID:

PHP

```
$id = $_GET["id"];
```

Depois:

PHP

```
$sql = "
    SELECT *
    FROM alunos
    WHERE id = ?
";
```

Utilizando prepared statement:

PHP

```
$stmt = mysqli_prepare(
    $conexao,
    $sql
);

mysqli_stmt_bind_param(
    $stmt,
    "i",
    $id
);

mysqli_stmt_execute($stmt);

$resultado =
    mysqli_stmt_get_result($stmt);

$aluno =
    mysqli_fetch_assoc($resultado);
```

19. Formulário de edição

HTML

```
<form
    action="atualizar.php"
    method="POST"
>

<input
```

```

        type="hidden"
        name="id"
        value="<?= $aluno["id"] ?>"
    >

    <div class="mb-3">

        <label class="form-label">
            Nome
        </label>

        <input

```

20. Atualizando

Crie:

atualizar.php

PHP

```

<?php

require_once "conexao.php";

$id = $_POST["id"];
$nome = $_POST["nome"];
$email = $_POST["email"];
$curso = $_POST["curso"];

$sql = "
    UPDATE alunos
    SET
        nome = ?,
        email = ?,
        curso = ?
    WHERE id = ?
";

$stmt = mysqli_prepare(
    $conexao,
    $sql
);

mysqli_stmt_bind_param(
    $stmt,
    "sssi",
    $nome,
    $email,

```

```
        $curso,  
        $id  
    );  
  
    mysqli_stmt_execute($stmt);  
  
    header("Location: index.php");  
  
    exit;
```

21. DELETE

Crie:

excluir.php

PHP

```
<?php  
  
require_once "conexao.php";  
  
$id = $_GET["id"];  
  
$sql = "  
    DELETE FROM alunos  
    WHERE id = ?  
";  
  
$stmt = mysqli_prepare(  
    $conexao,  
    $sql  
);  
  
mysqli_stmt_bind_param(  
    $stmt,  
    "i",  
    $id  
);  
  
mysqli_stmt_execute($stmt);  
  
header("Location: index.php");  
  
exit;
```


22. JavaScript

Agora vamos adicionar validação no navegador.

Crie:

script.js

JavaScript

```
const formulario =
  document.getElementById("formAluno");

formulario.addEventListener(
  "submit",
  function(event) {

    const nome =
      document.getElementById("nome");

    const email =
      document.getElementById("email");

    const curso =
      document.getElementById("curso");

    if (nome.value.trim() === "") {

      event.preventDefault();

      alert("Informe o nome.");

      nome.focus();

      return;
    }

    if (email.value.trim() === "") {

      event.preventDefault();

      alert("Informe o e-mail.");

      email.focus();

      return;
    }

    if (curso.value.trim() === "") {
```

```
        event.preventDefault();

        alert("Informe o curso.");

        curso.focus();

        return;
    }
}

);
```

23. Incluindo o JavaScript

Antes de fechar o `body`:

HTML

```
<script src="script.js"></script>
```

Agora temos:

```
Usuário
  ↓
JavaScript
  ↓
Validação
  ↓
POST
  ↓
PHP
  ↓
MySQL
```

Mas há um conceito muito importante:

JavaScript não substitui a validação PHP.

O JavaScript melhora a experiência do usuário, mas o servidor deve validar novamente os dados.

24. Confirmação de exclusão

No JavaScript:

JavaScript

```
function confirmarExclusao() {  
  
    return confirm(  
        "Deseja realmente excluir este aluno?"  
    );  
}
```

O botão:

HTML

```
<a  
    href="excluir.php?id=<?= $aluno["id"] ?>"  
    class="btn btn-danger btn-sm"  
    onclick="return confirmarExclusao()"  
>  
    Excluir  
</a>
```

25. CRUD completo

Agora podemos apresentar a mesma relação utilizada na aula Java:

Operação	SQL	PHP
Create	INSERT	mysqli_stmt_execute()
Read	SELECT	mysqli_query() / prepared statement
Update	UPDATE	mysqli_stmt_execute()
Delete	DELETE	mysqli_stmt_execute()

26. Comparação com Java

Esse ponto é particularmente interessante para os alunos:

Java Swing	PHP Web
JFrame	HTML
TextField	<input>

Java Swing

JButton

JTable

JOptionPane

JDBC

PreparedStatement

ResultSet

Evento Swing

Aplicação desktop

PHP Web

<button>

<table>

alert() / Bootstrap Modal

MySQLi

mysqli_prepare()

mysqli_fetch_assoc()

Evento JavaScript

Aplicação web

Isso permite mostrar que **CRUD é um conceito independente da linguagem.**

O que muda é a tecnologia utilizada para implementá-lo.

27. Exercício de fixação — Produtos

Criar:

SQL

```
CREATE TABLE produtos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  descricao VARCHAR(255),  
  preco DECIMAL(10,2),  
  estoque INT  
);
```

Interface:

Nome

Descrição

Preço

Estoque

[Cadastrar]

ID	Nome	Descrição	Preço	Estoque
----	------	-----------	-------	---------

Implementar:

- cadastrar;
- listar;
- editar;
- excluir;
- validar;
- confirmar exclusão.

28. Pesquisa

Adicionar:

HTML

```
<form method="GET">

  <div class="input-group">

    <input
      type="text"
      name="pesquisa"
      class="form-control"
      placeholder="Pesquisar aluno..."
    >

    <button
      class="btn btn-primary"
    >
      Pesquisar
    </button>

  </div>
```

No PHP:

PHP

```
$pesquisa = $_GET["pesquisa"] ?? "";
```

SQL:

SQL

```
SELECT *  
FROM alunos  
WHERE nome LIKE ?  
ORDER BY nome
```

Parâmetro:

PHP

```
$termo = "%" . $pesquisa . "%";  
  
mysqli_stmt_bind_param(  
    $stmt,  
    "s",  
    $termo  
);
```

29. Desafio final — Biblioteca

Criar:

```
categorias  
_____  
id  
nome
```

e:

```
livros  
_____  
id  
titulo  
autor  
ano  
isbn  
categoria_id
```

Relacionamento:

```
CATEGORIA  
|  
| 1:N  
|  
▼  
LIVROS
```

Consulta:

SQL

```
SELECT
    livros.id,
    livros.titulo,
    livros.autor,
    categorias.nome AS categoria
FROM livros
INNER JOIN categorias
    ON livros.categoria_id = categorias.id;
```

30. Evolução do projeto

Depois que o aluno dominar essa versão procedural, podemos evoluir:

```
PHP Procedural
    ↓
Funções
    ↓
Includes
    ↓
MySQLi
    ↓
CRUD
    ↓
Validação JS
    ↓
Bootstrap
    ↓
AJAX / Fetch
    ↓
PHP Orientado a Objetos
    ↓
MVC
    ↓
Laravel
```

Essa sequência é particularmente interessante para uma disciplina de **Desenvolvimento Web**, porque mostra ao aluno **por que** frameworks como Laravel existem.

31. Projeto final da aula

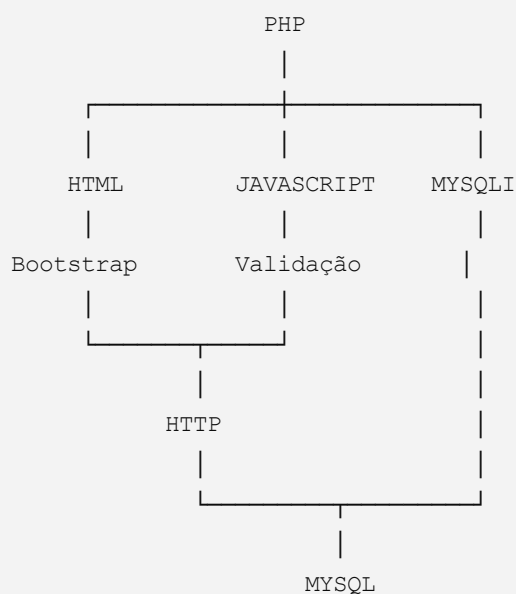
O aluno deverá entregar:

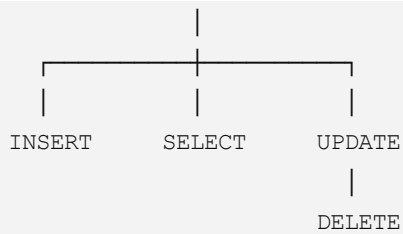
```
cadastro-alunos/
|
├─ index.php
├─ conexao.php
├─ salvar.php
├─ editar.php
├─ atualizar.php
├─ excluir.php
├─ script.js
|
└─ README.md
```

O sistema deverá possuir:

- Bootstrap;
- formulário;
- validação JavaScript;
- validação PHP;
- conexão MySQL;
- `INSERT` ;
- `SELECT` ;
- `UPDATE` ;
- `DELETE` ;
- pesquisa;
- confirmação de exclusão;
- mensagens para o usuário;
- código organizado.

Mapa conceitual





Sequência pedagógica

Eu manteria a mesma filosofia da aula Java: **primeiro SQL, depois PHP/MySQL, depois HTML/Bootstrap, depois JavaScript e, por fim, a integração completa.** Assim o aluno entende o fluxo da aplicação em vez de apenas copiar um CRUD pronto.